Affine Transformations – MATLAB

- In MATLAB, an affine transformation matrix in 2D looks like this:

$$T = \begin{bmatrix} a & d & 0 \\ b & e & 0 \\ c & f & 1 \end{bmatrix}$$

Affine Transformations – MATLAB

Translation	$\boldsymbol{P}' = \boldsymbol{P} \times \boldsymbol{T}$	$[x' \quad y' \quad 1] = [x \quad y \quad 1] \times \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix}$	t_x and t_y are translation factors
Rotation	$\boldsymbol{P}' = \boldsymbol{P} \times \boldsymbol{T}$	$[x' \quad y' \quad 1] = [x \quad y \quad 1] \times \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$	θ is the rotation angle (Clockwise)
Scaling	$\boldsymbol{P}' = \boldsymbol{P} \times \boldsymbol{T}$	$[x' \quad y' \quad 1] = [x \quad y \quad 1] \times \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$	s_x and s_y are scaling factors
Shearing	$\boldsymbol{P}' = \boldsymbol{P} \times \boldsymbol{T}$	$[x' \quad y' \quad 1] = [x \quad y \quad 1] \times \begin{bmatrix} 1 & sh_y & 0 \\ sh_x & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	sh_x and sh_y are shearing factors

Affine Transformations – MATLAB

- In MATLAB, a composite affine transformation means combining multiple affine operations—like translation, rotation, scaling, and shearing—into a single transformation matrix. This is done by matrix multiplication of individual transformation matrices in a specific order.
- In MATLAB, if we want to resize an image with scaling matrix $\boldsymbol{S}$, rotate it with rotation matrix $\boldsymbol{R}$, and move the result to some location with translation matrix $\boldsymbol{N}$, we simply form a 3×3 matrix equal to the product of the scaling, rotation, and translation matrices as follows:
- The composite representation for scaling, rotation, and translation (in that order):

$$\boldsymbol{T} = \boldsymbol{S} \times \boldsymbol{R} \times \boldsymbol{N}$$

Affine Transformations – MATLAB

- The composite representation for scaling, rotation, and translation (in that order):

$$\boldsymbol{T} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix}$$

- Thus, each pixel (x, y) in the input image is mapped to a new pixel (x', y') in the output image:

$$[x' \quad y' \quad 1] = [x \quad y \quad 1] \times \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix}$$